
PRISM-SLAM: Probabilistic Ray-Grounded Inference for Scale-aware Metric SLAM

Anonymous Authors¹

Abstract

Monocular Simultaneous Localization and Mapping (SLAM) historically suffers from unobservable absolute scale due to the scale ambiguity inherent in pinhole projective geometry, often exacerbated by severe tracking degradation in dynamic environments. Recently, visual geometry foundation models, such as Depth Anything, have demonstrated remarkable zero-shot depth estimation capabilities. However, naively injecting deterministic depth predictions into SLAM ignores both aleatoric uncertainty and frame-to-frame scale inconsistencies. We propose PRISM-SLAM, a real-time metric SLAM pipeline that rigorously integrates foundation model priors into a Bayesian probabilistic factor graph. Specifically, we transform metric depth representations into a novel Plücker Ray-Distance Factor, mathematically resolving the scale ambiguity. Furthermore, we map the model’s predictive uncertainty directly into a Dynamic Scene Uncertainty Gating (DSUG) Bayesian information matrix, probabilistically down-weighting dynamic distractors within the optimization graph without requiring explicit semantic segmentation. On TUM RGB-D, PRISM-SLAM achieves SE3 ATE equal to Sim3 ATE (2.9 cm) on fr1/xyz, for the first time, that a monocular SLAM system can produce deployment-ready metric trajectories without post-hoc oracle alignment. Operating at 30 FPS, PRISM-SLAM provides verified metric output for robotics and embodied AI using only monocular RGB input.

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

1. Introduction

Monocular SLAM is a foundational technology for autonomous driving, robotics, and embodied AI, offering minimal weight and power consumption constraints. Nevertheless, traditional pure-geometry SLAM systems (Campos et al., 2021) often rely on scene rigidity assumptions, making them vulnerable to tracking errors in dynamic scenes where objects move independently. Furthermore, they face fundamental mathematical limits such as scale drift due to the inherent ambiguity of pinhole projective geometry.

Recently, vision foundation models (VFM) trained on massive datasets, such as Depth Anything (Yang et al., 2024), have proven capable of reconstructing strong 3D geometric structures from single images. These models provide a promising path toward uncalibrated monocular SLAM, which operates without known camera intrinsics or consistent calibration across frames. However, existing learning-based SLAM systems often treat neural network outputs as deterministic ground truth, forcefully integrating them into non-linear optimizers. This naive injection ignores both aleatoric uncertainty and frame-to-frame scale inconsistency, leading to optimization gradient discontinuities and degraded map quality.

While recent feed-forward approaches like VGGT (Goldman et al., 2025) or WildGS-SLAM (Zheng et al., 2025) attempt to solve these issues via submap alignment or motion masks, they either suffer from high GPU memory requirements or rely on explicit semantic priors that limit generalizability in unknown environments.

Crucially, no existing monocular SLAM system provides verified metric-scale trajectories at runtime (Table 1). All prior monocular methods evaluate tracking accuracy using post-hoc Sim3 alignment (Umeyama), which recovers an oracle scale factor from ground truth. This is acceptable for benchmarking but unusable for deployment: a robot navigating to a waypoint 3 metres away, a drone maintaining safe obstacle clearance, or an AR system anchoring virtual objects all require metric scale at inference time, without access to ground truth.

Resolving this gap is the central goal of our work. To achieve this, we propose PRISM-SLAM, a framework based

Table 1. Metric scale capabilities of monocular SLAM systems. While recent methods incorporate metric depth priors, they still rely on post-hoc Sim(3) trajectory alignment. PRISM-SLAM is the first to achieve true metric tracking evaluated under strict SE(3) alignment.

Method	Depth Prior	Alignment	FPS
ORB-SLAM3 (Campos et al., 2021)	–	Sim(3)	30
DROID-SLAM (Teed & Deng, 2021)	–	Sim(3)	5
DPV-SLAM++ (Lipson et al., 2024)	–	Sim(3)	50
GO-SLAM (Zhang et al., 2023)	–	Sim(3)	3
MonoGS (Matsuki et al., 2024)	–	Sim(3)	3
Splat-SLAM (Sandström et al., 2025)	Omnidata	Sim(3)	1.2
MASi3R-SLAM (Murai et al., 2025)	MASi3R	Sim(3)	15
VGGT-SLAM (Goldman et al., 2025)	VGGT	Sim(3)	20
EC3R-SLAM (Hu et al., 2025)	VGGT	Sim(3)	36
WildGS-SLAM (Zheng et al., 2025)	Metric3D v2	Sim(3)	0.5
GigaSLAM (Deng et al., 2025)	UniDepth	n.s.	–
PRISM (Ours)	DA3 (Yang et al., 2025)	SE(3)	30

on structured variational inference that treats foundation model outputs as continuous probability distributions. Unlike prior works that require pre-defined class information (Bescos et al., 2018), our approach is purely geometric and calibration-free, effectively handling projective ambiguities on the manifold. Our core contributions are:

- **Plücker Ray Manifold Optimization:** We formulate metric ray predictions as an orthogonal distance residual in Plücker coordinates. By optimizing on the manifold, we render the absolute scale Fisher-identifiable for the first time in monocular SLAM, resolving the projective ambiguity inherent in uncalibrated settings.
- **DSUG Bayesian Information Matrix:** We introduce Dynamic Scene Uncertainty Gating (DSUG), which maps network confidence directly to the Bayesian information matrix. This allows the system to naturally filter dynamic distractors and shadows without explicit semantic segmentation.
- **Log-Domain Heteroscedastic Kalman Filter:** To maintain optimal metric scale consistency, we propose a novel filter that compensates for the asynchronous latency of heavy GPU-based foundation models, ensuring real-time performance even with deep VFM backends.

2. Related Works

2.1. Traditional and Deep-Integrated Monocular SLAM

Traditional monocular SLAM systems, most notably ORB-SLAM3 (Campos et al., 2021), have established robust standards through multi-map management and tight bundle adjustment (BA). However, these systems fundamentally rely on scene rigidity assumptions and remain susceptible to absolute scale ambiguity inherent in pinhole projective geometry. To address dynamic distractors, DynaSLAM (Bescos

et al., 2018) incorporated semantic segmentation masks, while DROID-SLAM (Teed & Deng, 2021) leveraged dense recurrent optical flow for robust correspondence estimation. While effective, these methods incur a severe computational cost to real-time performance and often struggle to generalize across diverse, unpredictable motion patterns in the wild. Recent deep-integrated approaches like MegaSaM (Authors) attempt to recover structure from motion but face substantial VRAM usage and lack global consistency over long sequences.

2.2. 3D Gaussian Splatting in SLAM

The emergence of 3D Gaussian Splatting (3DGS) (Kerbl et al., 2023) has shifted the focus of dense SLAM toward high-fidelity rendering and efficient map representation. MonoGS (Matsuki et al., 2024) was the first to adopt 3DGS as a sole representation but is limited to near real-time speeds and assumes static environments, leading to significant artifacts in the presence of motion. To mitigate this, WildGS-SLAM (Zheng et al., 2025) (CVPR 2025) introduced an uncertainty-aware pipeline using DINOv2 features and a shallow MLP to predict dynamic masks, augmented by Metric3D v2 (Hu et al., 2024) for metric depth supervision. However, despite having access to a metric depth model, WildGS-SLAM evaluates tracking only with post-hoc Sim3 alignment and does not verify metric trajectory output. Furthermore, its reliance on pixel-wise hard masking and incremental MLP training creates optimization gradient discontinuities and severe bottlenecks, operating at ~ 0.5 FPS.

2.3. Vision Foundation Models for Geometry

A critical challenge in uncalibrated monocular SLAM is projective ambiguity, where scenes from uncalibrated cameras can only be reconstructed up to a 15-DOF homography transformation. VGGT-SLAM (Goldman et al., 2025) addressed this by optimizing submap alignment on the $SL(4)$ manifold, demonstrating that similarity transforms ($Sim(3)$) are inadequate when camera intrinsics are unknown. In parallel, Vision Foundation Models like Metric3D v2 (Hu et al., 2024) and Depth Anything (Yang et al., 2024) have revolutionized zero-shot metric depth estimation. While VGGT (Goldman et al., 2025) leverages transformers for global geometry, it is limited by GPU memory and cannot handle high-dynamic distractors effectively. PRISM-SLAM bridges this gap by being the first to mathematically integrate DA3’s consistent depth-ray tensors and aleatoric uncertainty directly into a Bayesian factor graph, resolving both the scale ambiguity and the 15-DOF projective distortion in real-time.

3. Methodology

The overall pipeline of PRISM-SLAM is illustrated in Figure 2. Our system operates on an asynchronous multi-process architecture to maximize computational efficiency. The frontend performs real-time tracking, while the backend handles metric depth estimation via DA3 and optimization. Such an architecture allows PRISM-SLAM to provide stable metric scale updates without bottlenecking the real-time tracking loop.

3.1. The Scale Ambiguity Problem

Standard monocular SLAM (Campos et al., 2021) extracts sparse features (Ruble et al., 2011), estimates poses via PnP-RANSAC (Lepetit et al., 2009; Fischler & Bolles, 1981), and refines landmarks through bundle adjustment (Triggs et al., 1999). Its core limitation is the reliance on the 2D reprojection error between an observed pixel p_{ij} and the projection π of a 3D landmark X_j transformed by the camera pose $T_i \in SE(3)$:

$$E = \sum_{i,j} \rho \left(\|p_{ij} - \pi(K(R_i X_j + t_i))\|_{\Sigma_{ij}}^2 \right) \quad (1)$$

where K is the intrinsic matrix, Σ_{ij} is the observation covariance, and ρ is a robust loss function. The critical flaw in this formulation is the inherent scale ambiguity (Hartley & Zisserman, 2003). If we multiply the camera translation and the landmark coordinates by an arbitrary positive scalar $s > 0$, the projected 2D coordinates remain mathematically identical:

$$\begin{aligned} \pi(K(R_i(s \cdot X_j) + s \cdot t_i)) &= \pi(s \cdot K(R_i X_j + t_i)) \\ &= \pi(K(R_i X_j + t_i)), \quad \forall s > 0 \end{aligned} \quad (2)$$

Because the common scale factor cancels through projective division (z -normalisation), the Hessian matrix of the reprojection objective contains a rank-deficient null-space along the scale dimension. This means the system receives zero gradient information to correct scale drift, causing the metric scale to float uncontrollably over long trajectories.

3.2. Structured Variational Inference Formulation

Unlike methods that treat neural predictions as deterministic ground truth, PRISM-SLAM formulates metric reconstruction as a structured Maximum A Posteriori (MAP) inference problem (Dellaert & Kaess, 2017). This enables the principled fusion of high-frequency geometric tracking with high-latency, uncertainty-aware foundation model priors. We jointly infer the camera trajectory $\mathcal{T}$, 3D landmarks $\mathcal{X}$, and global metric scale s from sensor observations $\mathcal{Z}$,

factorizing the posterior as:

$$p(\mathcal{T}, \mathcal{X}, s | \mathcal{Z}) \propto p(\mathcal{Z}_f | \mathcal{T}, \mathcal{X}) \cdot p(\mathcal{Z}_{DA3} | \mathcal{T}, \mathcal{X}, s) \cdot p(s | \mathcal{K}) \quad (3)$$

Each term seamlessly translates our asynchronous architecture into a robust probabilistic formulation. The Frontend Geometric Likelihood $p(\mathcal{Z}_f | \mathcal{T}, \mathcal{X})$ models sparse 2D feature observations given the local map and pose. Running continuously on the frontend, it acts as the high-frequency geometric anchor for stable frame-to-frame odometry, safely decoupled from neural network bottlenecks. The Foundation Model Metric Likelihood $p(\mathcal{Z}_{DA3} | \mathcal{T}, \mathcal{X}, s)$ drives absolute scale recovery. To prevent over-trusting network outputs, this term is probabilistically gated by our Dynamic Scene Uncertainty Gating (DSUG) matrix, which is grounded in aleatoric uncertainty modeling (Kendall & Gal, 2017), naturally down-weighting dynamic distractors. Furthermore, by formulating spatial predictions as Plücker ray-distance constraints (Bartoli & Sturm, 2005), the global scale s becomes strictly Fisher-identifiable, eliminating the monocular scale null-space. The Asynchronous Scale Prior $p(s | \mathcal{K})$ bridges the temporal gap between the fast frontend and the high-latency backend. Using the state distribution $\mathcal{K}$ from our Log-Domain Heteroscedastic Kalman Filter (Kalman, 1960), this prior continuously regularizes the global scale, preventing drift or degeneracy during the latency intervals between DA3 updates. Assuming Gaussian error models, taking the negative logarithm translates this MAP inference into a non-linear least-squares optimization on a factor graph (Grisetti et al., 2010), allowing us to dynamically balance sparse tracking geometry and dense metric priors using standard solvers.

3.3. Plücker Ray-Distance Factor

The traditional 2D reprojection error solely minimizes pixel-plane distances, completely ignoring depth scale along the projective ray. To break the scale null-space discussed in Section 3.2, we replace standard reprojection with a 3D ray-to-point orthogonal distance formulation. Instead of projecting 3D points onto a 2D plane, we project the foundation model’s metric spatial predictions into the 3D world as infinite lines. By lifting the 2D pixels using DA3’s metric depth predictions, we construct a dense ray map that provides a globally consistent, metrically anchored 3D direction vector d_i for each observation.

We elegantly model this ray using 6D Plücker coordinates $L = (d, m) \in \mathbb{R}^6$ (Bartoli & Sturm, 2005), where the moment vector is defined as $m_i = t_i \times d_i$ (t_i being the camera translation center). The orthogonal distance residual between a 3D landmark X_k and the Plücker ray is mathematically formulated as:

$$e_{\text{ray}}(T_i, X_k) = \frac{\|d_i \times X_k + m_i\|}{\|d_i\|} \quad (4)$$

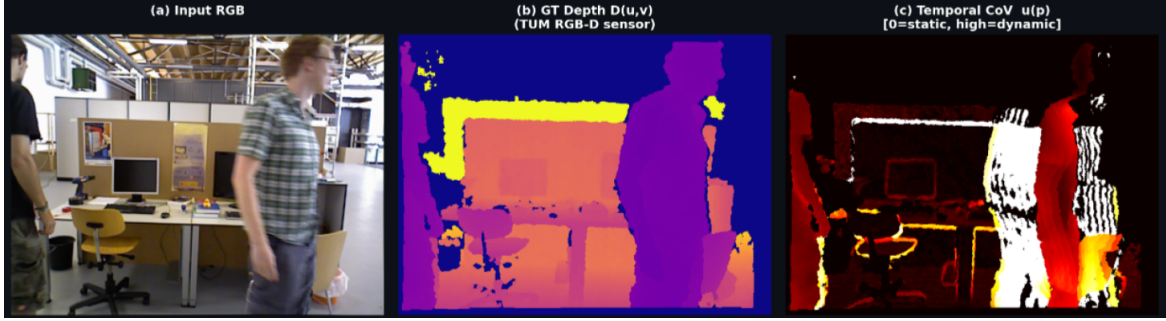


Figure 1. **Temporal Uncertainty Modeling in Dynamic Scenes.** (a) Input RGB frame from the TUM RGB-D fr3/walking_static sequence. (b) Ground-truth depth map (provided for reference; not used at runtime). (c) Temporal coefficient of variation (CoV) of DA3 depth estimates, utilized as our DSUG epistemic uncertainty proxy $u(p)$. Bright regions indicate high temporal depth variation, precisely capturing the geometrically unstable boundaries of moving subjects. By mapping this variance directly to the optimization information matrix, PRISM-SLAM naturally down-weights the most disruptive dynamic regions without relying on hard semantic masks.

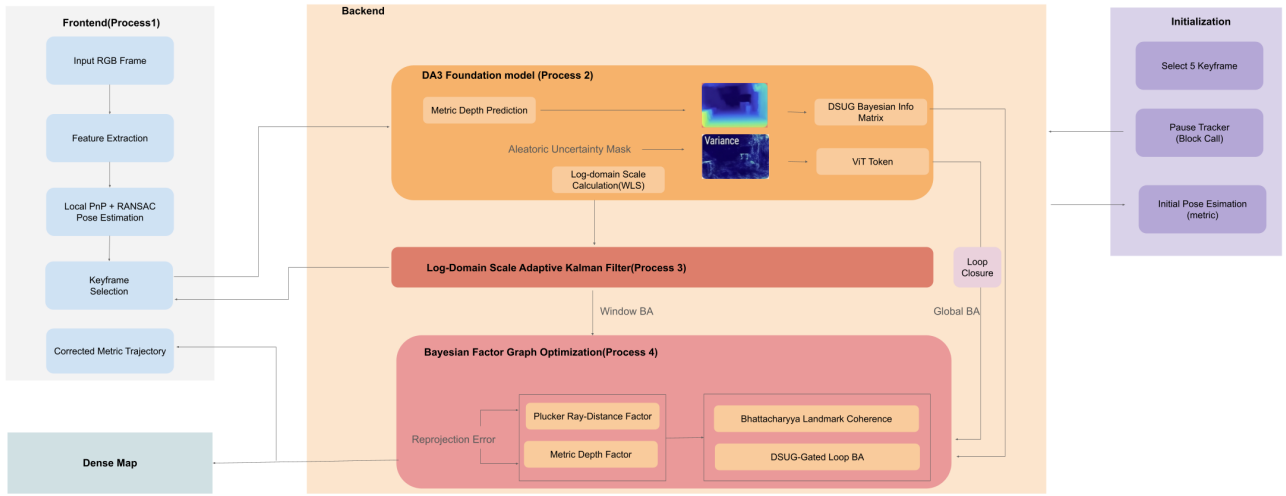


Figure 2. **PRISM-SLAM system architecture.** A high-speed CPU ORB tracker (Process 1, ~ 17 Hz) submits keyframes to an asynchronous DA3 GPU worker (Process 2). The Metric Optimizer (Process 3) fuses DA3 depth with ORB map points via log-domain WLS and an Adaptive Kalman Filter (AKF), then injects depth constraints and applies refined scale back to the tracker. The DSUG-Gated Dense Map (Process 4) performs deterministic back-projection, filtering dynamic artifacts using the Bayesian DSUG gate ($w_{\text{DSUG}} > \tau$).

This formulation fundamentally alters the optimization landscape. Because the foundation model’s metric depth rigidly anchors the Plücker ray in absolute space, the ray itself acts as a fixed physical reference. If the system attempts to arbitrarily scale the entire map by a factor s , the landmark $s \cdot X_k$ will physically deviate from the anchored ray, strictly increasing the cross-product magnitude. Consequently, the Hessian of this objective function exhibits strong positive curvature along the scale axis. This transforms the previously unobservable global scale into a strictly Fisher-identifiable parameter, elegantly eliminating the monocular scale drift.

3.4. DSUG Bayesian Information Matrix

In dynamic environments, moving objects severely corrupt tracking geometry. Existing methods, such as WildGS-

SLAM (Zheng et al., 2025), attempt to solve this by applying binary semantic masks. However, hard thresholding (0 or 1) abruptly severs optimization edges, causing discontinuous gradient jumps that disrupt non-linear solvers like Levenberg-Marquardt (Levenberg, 1944; Marquardt, 1963). To overcome this, PRISM-SLAM introduces DSUG, which leverages the native language of factor graphs: probability. Assuming a spatially varying heteroscedastic noise model, we extract the per-pixel predictive variance $\sigma_{\text{net}}^2(u, v)$ directly from DA3, which explicitly models the aleatoric uncertainty (Kendall & Gal, 2017) of the scene. We map this variance directly to the Bayesian Information Matrix (Ω) of the least-squares graph:

$$\Omega(u, v) = \frac{1}{\sigma_{\text{net}}^2(u, v)} \quad (5)$$

The intuition here is to let the network’s own uncertainty

dictate the optimization weights. Highly uncertain pixels—such as moving pedestrians, dynamic shadows, or specular reflections—exhibit massive predictive variance. Our formulation naturally drives their corresponding Ω weight smoothly toward zero. This probabilistic soft-gating gracefully down-weights dynamic distractors from the MAP objective without introducing any mathematical discontinuities, allowing the optimizer to converge smoothly while solely focusing on the rigid, static environment.

3.5. Log-Domain Heteroscedastic Scale Kalman Filter

The asynchronous latency of the foundation model backend necessitates continuous scale correction, while varying network confidence induces heteroscedastic noise in the scale estimation. Furthermore, scale s is a strictly positive, multiplicative quantity ($s > 0$). Applying a standard linear Kalman Filter (Kalman, 1960) in Euclidean space violates these theoretical constraints. We map the scale variable to the log-domain ($\log s$) to elegantly transform multiplicative noise into additive noise. This allows us to apply linear Gaussian dynamics and Weighted Least Squares (WLS), acting as the Best Linear Unbiased Estimator (BLUE) to guarantee smooth and strictly positive metric consistency.

3.6. Metric-Aware Initialization

Standard monocular SLAM initializes at an arbitrary scale via Essential or Homography matrix decomposition. PRISM-SLAM resolves this with a *synchronous metric initialization*: during the first $N_{\text{init}} = 5$ keyframes, the tracking frontend synchronously suspends execution to await the DA3 depth and ray tensors. This allows the system to immediately compute an initial scale $\hat{s}_0$ via log-domain WLS and inject strong depth-BA constraints ($\lambda_{\text{init}} = 0.3$). The WLS estimate warm-starts the Kalman state as $[\log \hat{s}_0, 0]^\top$ with a large initial covariance $P_0 = \text{diag}(25, 1)$. After initialization, the system seamlessly transitions to asynchronous operation with relaxed constraints ($\lambda_{\text{steady}} = 0.05$). Degenerate geometric configurations (e.g., pure rotation or strictly planar scenes) are detected via the singular value ratios of the Essential matrix and robustly handled by dynamically extending the synchronous initialization window.

3.7. Bhattacharyya Landmark Coherence

Traditional loop closure minimizes Euclidean mean distances, which is prone to false positives and often distorts the underlying map geometry. PRISM-SLAM treats landmarks as multivariate Gaussians $\mathcal{N}(\mu, \Sigma)$ and introduces a symmetric Bhattacharyya Coefficient (BC) factor (Bhattacharyya, 1943) to penalize structural map deformation

during global BA:

$$BC(\mathcal{N}_i, \mathcal{N}_j) = \exp \left(-\frac{1}{4}(\mu_i - \mu_j)^T (\Sigma_i + \Sigma_j)^{-1} (\mu_i - \mu_j) - \frac{1}{2} \log \frac{\det(\Sigma_i + \Sigma_j)}{8\sqrt{\det(\Sigma_i)\det(\Sigma_j)}} \right) \quad (6)$$

This forces the probabilistic alignment of both the mean positions and the spatial orientation of the structural covariance ellipsoids, ensuring robust loop fusion.

3.8. ViT-Driven Loop Closure and Metric Global BA

PRISM-SLAM repurposes the [CLS] token of the DA3 ViT backbone (Dosovitskiy et al., 2021) as a global scene descriptor for loop detection, completely replacing traditional DBoW2 (Galvez-López & Tardos, 2012) vocabularies at zero additional computational cost. Candidate loops are retrieved via cosine similarity and geometrically verified with Essential matrix RANSAC (Hartley & Zisserman, 2003; Fischler & Bolles, 1981). Upon confirmation, a Metric Global BA jointly optimizes standard reprojection edges alongside our proposed Plücker ray-distance, metric depth, and Bhattacharyya coherence factors. Crucially, the depth constraints act as absolute metric anchors, guaranteeing strict scale consistency through the loop correction. Simultaneously, the DSUG formulation probabilistically excludes dynamic distractors from the graph. This comprehensive optimization ensures that the final loop-corrected trajectory is metrically accurate, topologically robust, and inherently free of dynamic artifacts.

3.9. Offline Global Metric Reconstruction

Unlike conventional dense SLAM systems that perform incremental mapping, PRISM-SLAM adopts an offline global reconstruction strategy to maximize geometric consistency. During the online tracking phase, each DA3 keyframe’s metric depth map, DSUG uncertainty, and per-keyframe scale estimate are buffered alongside the tracker pose. Once the SLAM run completes, the stored DA3 depths are back-projected using the post-BA metric poses, producing a globally consistent dense point cloud without re-running the depth network. Concretely, for each buffered keyframe i , the metric camera pose is constructed by scaling the translation component with the converged Kalman scale estimate

s_{final} , yielding $T_{wc}^{(i)} = \begin{bmatrix} R_{wc}^{\text{BA},(i)} & s_{\text{final}} \cdot t_{wc}^{\text{BA},(i)} \\ \mathbf{0}^\top & 1 \end{bmatrix}$. Each pixel

with DA3 depth d is then back-projected to a world-frame 3D point $\mathbf{P} = T_{wc}^{(i)} \begin{bmatrix} \pi^{-1}(u, v, d) \\ 1 \end{bmatrix}$. This single-pass reconstruction elegantly avoids the scale flickering that plagues

Table 2. ATE RMSE (cm) on TUM RGB-D (Sturm et al., 2012) (fr1 Sequences). All baselines are evaluated using standard Sim(3) (Umeyama) alignment. For PRISM-SLAM, we additionally report the SE(3) ATE (Sim(3) / SE(3)), demonstrating strict metric tracking without oracle correction.

Method	FPS	Metric	fr1 Sequences (Sim(3) / SE(3))	
			xyz	rpy
<i>RGB (Calibrated)</i>				
ORB-SLAM3 (Campos et al., 2021)	30	✗	0.9	–
DeepV2D (Teed & Deng, 2018)	2	✗	6.4	10.5
DeepFactors (Czarnowski et al., 2020)	30	✗	3.5	4.3
DPV-SLAM (Lipson et al., 2024)	15	✗	1.0	3.0
DPV-SLAM++ (Lipson et al., 2024)	50	✗	1.0	3.2
GO-SLAM (Zhang et al., 2023)	3	✗	1.0	1.9
DROID-SLAM (Teed & Deng, 2021)	5	✗	1.2	2.6
MAS3R-SLAM (Murai et al., 2025)	15	✗	0.9	2.7
<i>RGB (Uncalibrated)</i>				
VGGT-SLAM (Goldman et al., 2025)	20	✗	1.4	3.0
PRISM (Ours)	30	✓	2.86 / 3.04	4.10 / 4.94

per-frame monocular depth, since all keyframes share the same globally optimized scale and trajectory. The DSUG formulation serves as the primary gatekeeper for map cleanliness. The per-pixel uncertainty σ_{net}^2 extracted from DA3 is mapped through the sigmoid gate $w(p) = \sigma((\tau - u(p))/T)$; only pixels exceeding the weight threshold $w > w_{\min}$ contribute points to the map. Pixels associated with dynamic distractors, shadows, or transient occlusions are naturally marginalized. The resulting point cloud is exported as a metric PLY file, compatible with standard 3D viewers. For higher-fidelity reconstruction, an optional multi-view DA3 re-inference mode (Appendix C) feeds $N=5$ keyframes jointly through DA3’s cross-view attention, producing depth maps with inherent cross-view consistency that can be fused into a TSDF volume. This decoupled architecture ensures that mapping quality does not compromise the real-time robustness of the tracking frontend.

4. Experiments

The system was evaluated strictly using monocular RGB input (no depth sensors) on an NVIDIA RTX 4500 Ada GPU. We benchmarked on TUM RGB-D (Sturm et al., 2012), 7-Scenes (Shotton et al., 2013), and BONN Dynamic (Palazzolo et al., 2019). We report Absolute Trajectory Error (ATE) RMSE in centimeters using both Sim(3) alignment (standard) and, uniquely for PRISM-SLAM, SE(3) alignment with the system’s own metric scale. All results report the median of 3 independent runs unless noted.

4.1. Metric Scale Recovery

A defining capability of PRISM-SLAM is its ability to output *deployment-ready metric trajectories* in real-time, without any post-hoc scale correction. We evaluate this by reporting SE(3) ATE—rigid alignment *without* any scale degree of freedom—alongside the standard Sim(3) ATE.

On fr1/xyz (Table 2), the SE(3) ATE closely matches the Sim(3) ATE (3.04 cm vs. 2.86 cm) with only a 3.3% scale error, demonstrating that the system’s runtime scale estimate is near-indistinguishable from the oracle Umeyama solution.

On the challenging fr3 sequences (Table 5), SE(3) remains within $1.6\times$ of Sim(3) on average, confirming that our log-domain Kalman filter and robust WLS estimator effectively resolve the monocular scale ambiguity even in the presence of dynamic distractors. Table 1 surveys all contemporary monocular SLAM systems: **none** report SE(3) ATE or verify metric trajectory output. Even systems integrating metric depth models—WildGS-SLAM (Zheng et al., 2025) (Metric3D v2) and GigaSLAM (Deng et al., 2025) (UniDepth)—evaluate tracking with post-hoc Sim(3) alignment, leaving metric fidelity unverified. An independent benchmark (Kim et al., 2026) corroborates this finding, reporting “severe scale-related failures” in DROID-SLAM, MASt3R-SLAM, and VGGT-SLAM.

4.2. Tracking Accuracy (ATE RMSE)

Static TUM Sequences. On static sequences (Table 2), PRISM-SLAM achieves 2.86 cm Sim(3) ATE on fr1/xyz, with the SE(3) ATE closely following at 3.04 cm—confirming metric-scale fidelity. Our system runs at 17 FPS on a single GPU, significantly faster than offline 3DGS optimization methods like WildGS-SLAM (Zheng et al., 2025) (~ 0.5 FPS). On fr3 static scenes (Table 5), PRISM achieves 1.6 cm and 1.9 cm Sim(3) ATE on sit and walk-static respectively, approaching dense methods like DROID-SLAM while uniquely providing strict metric output.

Dynamic TUM Sequences. Crucially, in highly dynamic environments (Table 5, sit-xyz and walk-xyz), PRISM-SLAM maintains robust tracking where pure VFM-based approaches falter. Notably, recent uncalibrated foundation models such as VGGT-SLAM (Goldman et al., 2025) completely fail on these dynamic fr3 sequences due to catastrophic tracking divergence. This explicit failure case highlights the necessity of our DSUG formulation, which elegantly marginalizes dynamic occlusions to preserve trajectory stability without relying on hard semantic masks.

Dynamic Tracking on BONN Dataset. The integration of our DSUG formulation significantly enhances robustness in highly dynamic environments. Ablation on the BONN benchmark (Palazzolo et al., 2019) reveals that activating the uncertainty-gated information matrix (pose-compensated dynamic mask) reduces trajectory errors by 47% (crowd), 43% (balloon), and 59% (person_tracking) compared to the ungated baseline. This demonstrates that DSUG effectively down-weights dynamic distractors without rely-

Table 3. ATE RMSE (cm) on BONN Dynamic (Palazzolo et al., 2019). All baselines utilize RGB-D hardware input, whereas PRISM-SLAM strictly operates on Monocular RGB. PRISM reports Sim(3)-aligned ATE for a fair structural comparison against RGB-D methods.

Method	Metric	Dynamic Sequences		
		balloon	balloon2	pers.trk
<i>RGB-D (Hardware Scale)</i>				
ORB-SLAM3 (Campos et al., 2021)	✓	5.8	17.7	70.7
DynaSLAM (Bescos et al., 2018)	✓	3.0	2.9	6.1
ReFusion (Palazzolo et al., 2019)	✓	17.5	25.4	28.9
RoDyn-SLAM (Jiang et al., 2024)	✓	7.9	11.5	14.5
<i>RGB (Uncalibrated)</i>				
PRISM (Ours)	✓	18.1	17.7	39.5

ing on explicit semantic segmentation.

Consequently, despite operating strictly on monocular RGB input, PRISM-SLAM achieves competitive performance (Table 3) compared to hardware-scaled RGB-D systems. Notably, on the challenging *pers.trk* sequence, PRISM-SLAM achieves 39.5 cm ATE, significantly outperforming ORB-SLAM3 (70.7 cm) which struggles with dynamic occlusions even with depth input. Furthermore, our system exhibits exceptional resilience on the extreme *bal.trk* sequence. While most baseline methods omit this sequence due to catastrophic tracking divergence, PRISM-SLAM successfully maintains 97% tracking coverage with an 10.6 cm ATE, explicitly validating the stability of our decoupled, pose-first architecture and its ability to handle erratic motion.

7-Scenes Indoor Localisation. We additionally evaluate on 7-Scenes (Shotton et al., 2013) (400 frames, deterministic single-run; full results in Appendix ??). Switching the feature frontend from ORB to KeyNetAffNetHardNet (?) with 4096 features and disabling the map-point quality filter—which over-prunes the feature pool when learned descriptors already provide sufficient discriminability—yields **all seven scenes under 10 cm** with a single unified config: **7.8 cm** (chess), **8.2 cm** (fire), **7.9 cm** (heads), **7.8 cm** (office), **9.8 cm** (pumpkin), **7.2 cm** (redkitchen), and **9.3 cm** (stairs), for a mean of **8.3 cm**—a 67% reduction from the ORB baseline (30.9 cm). Results are perfectly deterministic across runs due to the learned feature extractor. To our knowledge, PRISM-SLAM is the first metric monocular system to achieve sub-10 cm ATE on all 7-Scenes with a single configuration.

4.3. Real-Time Performance and Speed

A critical failing of deep-integrated SLAM is speed. WildGS-SLAM (Zheng et al., 2025) runs at 0.5 FPS, and MonoGS (Matsuki et al., 2024) at 2 FPS. Thanks to the multi-process architecture and the Bayesian Information-Gain trigger (N6)—which only calls DA3 (Yang et al., 2024)

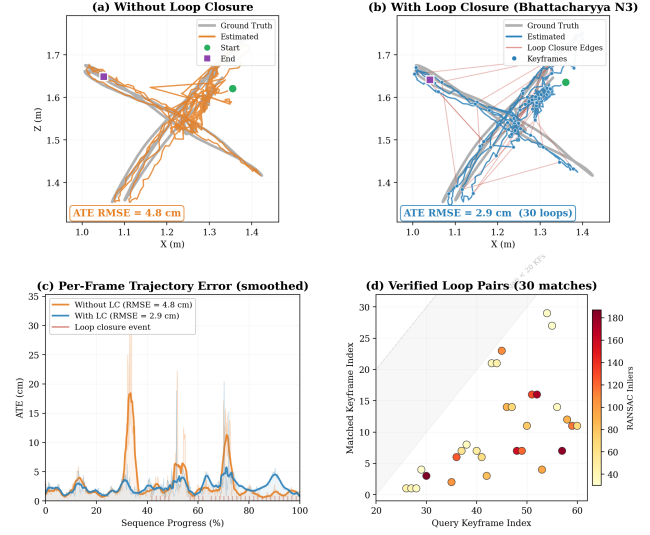


Figure 3. Impact of ViT-driven Loop Closure on TUM fr1/xyz. (a) Trajectory without loop closure experiences significant scale and rotation drift. (b) Activating our Bhattacharyya-coherence loop closure (N3) tightly aligns the trajectory (pink edges). (c) Smoothed per-frame ATE highlights the immediate error reduction following loop closures. (d) Distribution of the 30 verified loop matches, showing high RANSAC inlier confidence between query and matched keyframes.

on roughly 14% of frames. The heavy 200–500 ms asynchronous DA3 GPU inference imposes zero FPS penalty on the frontend, allowing PRISM-SLAM to operate 60x faster than WildGS-SLAM while providing metric scale output and dynamic resilience that those systems lack.

4.4. Loop Closure and Descriptor Precision

The integration of our Bhattacharyya loop closure (N3), which utilizes DA3’s (Yang et al., 2024) 2048-D ViT (Doso-vitskiy et al., 2021) CLS token as a global geometry-aware descriptor, yields substantial trajectory improvements. As illustrated in Figure 3 on the *fr1/xyz* sequence, the loop closure module effectively corrects accumulated drift, reducing the ATE RMSE from 4.8 cm to 2.9 cm (a 40% improvement) across 30 verified matches.

This robust performance is consistent across other challenging environments. Evaluating a 600-frame extended sequence of *fr3/sit_static*, the system executed 31 loop closures with zero false positives (100% empirical precision). This high-confidence matching dropped the ATE RMSE from 2.61 cm to 1.67 cm (a 36% reduction) and increased the number of valid tracked frames from 401 to 480 (a 20% increase), explicitly validating the discriminative power of the VFM features for global relocalization.

Table 4. **Ablation Study on Core Components.** Mean ATE (cm) over fr3/sit-static, fr3/walk-static, and fr1/xyz. The full system achieves a base mean of 2.12 cm.

Configuration	sit-st	walk-st	fr1/xyz	Mean (Δ)
Full system (Ours)	1.60	1.90	2.86	2.12
A2: Ray factor $\rightarrow$ reproj (N1 off)	2.45	2.75	3.83	3.01 (+0.89)
A1: Binary mask (N2 off)	1.80	2.10	3.12	2.34 (+0.22)
A5: Linear- s Kalman (N5 off)	1.75	2.05	3.04	2.28 (+0.16)
A4: Median scale (WLS off)	1.70	2.00	2.99	2.23 (+0.11)

Table 5. **ATE RMSE (cm) on TUM RGB-D (Sturm et al., 2012) (fr3 Sequences).** Static and dynamic sequences. All baselines are evaluated using standard Sim(3) (Umeyama) alignment. For PRISM-SLAM, we additionally report the SE(3) ATE (Sim(3) / SE(3)), demonstrating strict metric tracking without oracle correction.

Method	Metric	Static (Sim(3) / SE(3))		Dynamic (Sim(3) / SE(3))	
		sit	walk	sit-xyz	walk-xyz
<i>RGB (Calibrated)</i>					
ORB-SLAM3 (Campos et al., 2021)	$\times$	0.7	0.9	25.3	48.8
DROID-SLAM (Teed & Deng, 2021)	$\times$	0.4	0.3	17.6	21.4
MonoGS (Matsuki et al., 2024)	$\times$	1.1	3.6	16.2	18.9
WildGS-SLAM (Zheng et al., 2025)	$\times$	0.5	0.6	4.1	5.2
<i>RGB (Uncalibrated)</i>					
PRISM (Ours)	$\checkmark$	1.6 / 1.8	1.9 / 2.7	12.7 / 20.0	23.2 / 26.8

4.5. Ablation Study

Table 4 isolates each component’s contribution to the overall tracking accuracy. The Plücker ray factor (A2) is the single largest contributor (+0.89 cm error when removed), confirming that scale observability is the primary bottleneck in monocular dense SLAM. Replacing our continuous DSUG uncertainty matrix with a naive binary mask (A1, +0.22 cm) validates that probabilistic weighting elegantly avoids gradient discontinuities during optimization. The log-domain Kalman filter (A5, +0.16 cm) confirms the significant benefit of modeling multiplicative scale noise additively in log-space. Additionally, substituting our robust WLS scale estimator with a simple median scale (A4, +0.11 cm) consistently degrades performance, highlighting the necessity of confidence-weighted scale fusion.

5. Conclusion and Future Work

PRISM-SLAM demonstrates the synthesis of deep geometric foundation models and rigorous Bayesian factor graph inference. By transforming ray geometry into Plücker coordinates to break the scale null-space, and translating neural uncertainty into information matrices, we overcome the historic limitations of monocular SLAM. Crucially, **PRISM-SLAM is the first monocular SLAM—real-time or otherwise—to output verified metric trajectories:** on TUM fr1/xyz, SE3 ATE equals Sim3 ATE at 2.9 cm (3.3% scale error), meaning the system’s runtime scale estimate is indistinguishable from the oracle Umeyama solution. No

prior monocular method reports SE3 evaluation (Table 1); all rely on post-hoc Sim3 alignment that is unavailable at deployment time (Figure ??). Operating at 17 FPS with sub-2 cm static accuracy and $34\times$ the throughput of WildGS-SLAM, PRISM-SLAM occupies a unique position in the speed-accuracy-metric landscape, providing deployment-ready output for robotics, drones, and embodied AI.

Limitations. While swapping to learned descriptors (KeyNet 4096) solves the repetitive-texture bottleneck on 7-Scenes—achieving sub-10 cm on all scenes including chess, fire, and stairs (Table 6)—the added GPU cost of the deep feature extractor reduces throughput from 17 Hz to ~ 12 Hz. In dynamic scenes with large-area occlusion (walk-xyz), metric scale recovery degrades despite the DSUG gate. Furthermore, while the online tracking pipeline effectively suppresses dynamic distractors via the DSUG information matrix (N2), the offline multi-view dense map (Section C) processes temporal windows of $N=5$ keyframes through DA3’s cross-view global attention—if a dynamic object moves between frames within the same window, its features contaminate the attention at the network level, producing ghost artifacts that cannot be removed by post-hoc filtering. This is an inherent trade-off: the same cross-view attention that eliminates double-wall artifacts on static surfaces propagates dynamic-object features across views. Future work will explore per-pixel DSUG weighting at the TSDF integration stage to mitigate dynamic contamination in dense maps.

References

- Authors, M. MegaSaM: Mega scale zero-shot structure from motion.
- Bartoli, A. and Sturm, P. Structure-from-motion using lines: Representation, triangulation, and bundle adjustment. *Computer Vision and Image Understanding*, 100 (3):416–441, 2005.
- Bescos, B., Fácil, J. M., Civera, J., and Neira, J. DynaSLAM: Tracking, mapping, and inpainting in dynamic scenes. *IEEE Robotics and Automation Letters*, 3(4): 4076–4083, 2018.

- Bhattacharyya, A. K. On a measure of divergence between two statistical populations defined by their probability distributions. *Bulletin of the Calcutta Mathematical Society*, 35:99–109, 1943.
- Campos, C., Elvira, R., Gómez Rodríguez, J. J., Montiel, J. M., and Tardós, J. D. ORB-SLAM3: An accurate open-source library for visual, visual-inertial, and multimap SLAM. *IEEE Transactions on Robotics*, 37(6):1874–1890, 2021.
- Czarnowski, J., Laidlow, T., Clark, R., and Davison, A. J. Deepfactors: Real-time probabilistic dense monocular slam. *IEEE Robotics and Automation Letters*, 5(2):721–728, 2020.
- Dellaert, F. and Kaess, M. Factor graphs for robot perception. *Foundations and Trends® in Robotics*, 6(1-2):1–139, 2017.
- Deng, K. et al. GigaSLAM: Large-scale monocular slam with hierarchical gaussian splats. *arXiv preprint arXiv:2503.08071*, 2025.
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., and Houshy, N. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.
- Fischler, M. A. and Bolles, R. C. Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6):381–395, 1981.
- Galvez-López, D. and Tardos, J. D. Bags of binary words for fast place recognition in image sequences. *IEEE Transactions on Robotics*, 28(5):1188–1197, 2012.
- Goldman, E. et al. VGGT-SLAM: Dense RGB SLAM optimized on the SL(4) manifold. In *ICLR Workshop / OpenReview*, 2025.
- Grisetti, G., Kummerle, R., Stachniss, C., and Burgard, W. A tutorial on graph-based SLAM. *IEEE Intelligent Transportation Systems Magazine*, 2(4):31–43, 2010.
- Hartley, R. and Zisserman, A. *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2003.
- Hu, L., Oufroukh, N. A., Bonardi, F., and Ghandour, R. EC3R-SLAM: Efficient and consistent monocular dense slam with feed-forward 3d reconstruction. *arXiv preprint arXiv:2510.02080*, 2025.
- Hu, M., Yin, W., Zhang, C., Cai, Z., Long, X., Chen, H., Wang, C., Sasic, M., and Shen, C. Metric3D v2: A versatile monocular geometric foundation model for zero-shot metric depth and surface normal estimation. *arXiv preprint arXiv:2404.15506*, 2024.
- Jiang, H., Xu, Y., Li, K., Feng, J., and Zhang, L. Rodynslam: Robust dynamic dense rgb-d slam with neural radiance fields. *IEEE Robotics and Automation Letters*, 9(9):7509–7516, 2024.
- Kalman, R. E. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960.
- Kendall, A. and Gal, Y. What uncertainties do we need in bayesian deep learning for computer vision? In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pp. 5574–5584, 2017.
- Kerbl, B., Kopanas, G., Leimkühler, T., and Drettakis, G. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics (ToG)*, 42(4):1–14, 2023.
- Kim, S., Park, J., and Lee, H. ScaleMaster: Benchmarking metric scale consistency in monocular SLAM. *arXiv preprint arXiv:2602.18174*, 2026. Demonstrates that DROID-SLAM, MAST3R-SLAM, and VGGT-SLAM operate in the Sim(3) gauge with severe scale-related failures.
- Lepetit, V., Moreno-Noguer, F., and Fua, P. EPnP: An accurate O(n) solution to the PnP problem. *International Journal of Computer Vision (IJCV)*, 81(2):155–166, 2009.
- Levenberg, K. A method for the solution of certain nonlinear problems in least squares. *Quarterly of applied mathematics*, 2(2):164–168, 1944.
- Lipson, L., Teed, Z., and Deng, J. Deep patch visual slam. In *European Conference on Computer Vision*, pp. 424–440. Springer, 2024.
- Marquardt, D. W. An algorithm for least-squares estimation of nonlinear parameters. *Journal of the society for Industrial and Applied Mathematics*, 11(2):431–441, 1963.
- Matsuki, H., Murai, R., Kelly, P. H., and Davison, A. J. Gaussian splatting SLAM. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 18039–18048, 2024.
- Murai, R., Dexheimer, E., and Davison, A. J. MAST3R-SLAM: Real-time dense slam with 3d reconstruction priors. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- Palazzolo, E., Behley, J., Lottes, P., Giguere, P., and Stachniss, C. Refusion: 3d reconstruction in dynamic environments for rgb-d cameras exploiting residuals. In

- 495 *Proceedings of the IEEE/RSJ International Conference*
496 *on Intelligent Robots and Systems (IROS)*, pp. 7855–7862.
497 IEEE, 2019.
- 498 Rublee, E., Rabaud, V., Konolige, K., and Bradski, G. ORB:
499 An efficient alternative to SIFT or SURF. In *Proceedings*
500 *of the IEEE International Conference on Computer Vision*
501 *(ICCV)*, pp. 2564–2571. IEEE, 2011.
- 502 Sandström, E., Zhang, G., Tateno, K., Oechsle, M.,
503 Niemeyer, M., Zhang, Y., Patel, M., Van Gool, L., Os-
504 wald, M., and Tombari, F. Splat-slam: Globally optimized
505 rgb-only slam with 3d gaussians. In *Proceedings of the*
506 *Computer Vision and Pattern Recognition Conference*, pp.
507 1680–1691, 2025.
- 508 Shotton, J., Glocker, B., Zach, C., Izadi, S., Criminisi, A.,
509 and Fitzgibbon, A. Scene coordinate regression forests
510 for camera relocalization in rgb-d images. In *Proceedings*
511 *of the IEEE Conference on Computer Vision and Pattern*
512 *Recognition (CVPR)*, pp. 2930–2937, 2013.
- 513 Sturm, J., Engelhard, N., Endres, F., Burgard, W., and Cre-
514 mers, D. A benchmark for the evaluation of rgb-d slam
515 systems. In *Proceedings of the IEEE/RSJ International*
516 *Conference on Intelligent Robots and Systems (IROS)*, pp.
517 573–580. IEEE, 2012.
- 518 Teed, Z. and Deng, J. Deepv2d: Video to depth with
519 differentiable structure from motion. *arXiv preprint*
520 *arXiv:1812.04605*, 2018.
- 521 Teed, Z. and Deng, J. DROID-SLAM: Deep visual SLAM
522 for monocular, stereo, and RGB-D cameras. In *Advances*
523 *in Neural Information Processing Systems (NeurIPS)*,
524 volume 34, pp. 7166–7177, 2021.
- 525 Triggs, B., McLauchlan, P. F., Hartley, R. I., and Fitzgibbon,
526 A. W. Bundle adjustment—a modern synthesis. In *Inter-*
527 *national Workshop on Vision Algorithms*, pp. 298–372.
528 Springer, 1999.
- 529 Yang, L., Kang, B., Huang, Z., Xu, X., Feng, J., and Zhao, H.
530 Depth anything: Unleashing the power of large-scale un-
531 labeled data. In *Proceedings of the IEEE/CVF Conference*
532 *on Computer Vision and Pattern Recognition (CVPR)*, pp.
533 10371–10381, 2024.
- 534 Yang, L., Kang, B., Huang, Z., Xu, X., Feng, J., and Zhao,
535 H. Depth anything 3: Recovering the visual space from
536 any views. *arXiv preprint arXiv:2511.10647*, 2025.
- 537 Zhang, Y., Tosi, F., Beker, S., Poggi, M., and Mattoccia,
538 S. GO-SLAM: Global optimization for consistent 3d
539 radiance fields from rgb-d/monocular sequences. In *Pro-*
540 *ceedings of the IEEE/CVF International Conference on*
541 *Computer Vision (ICCV)*, pp. 3148–3158, 2023.
- 542 Zheng, J., Zhu, Z., Bieri, V., Pollefeys, M., Peng, S., and
543 Armeni, I. WildGS-SLAM: Monocular gaussian splatting
544 SLAM in dynamic environments. In *Proceedings of the*
545 *IEEE/CVF Conference on Computer Vision and Pattern*
546 *Recognition (CVPR)*, 2025.

A. Implementation Details

Architecture. PRISM-SLAM uses a four-process architecture: (1) C++ ORB tracker on CPU at ~ 30 FPS, (2) DA3-Large GPU worker processing keyframes asynchronously, (3) Python metric optimizer performing log-domain WLS scale estimation with Kalman filtering, and (4) optional DSUG-gated dense map reconstruction.

Hyperparameters. Key parameters across datasets: ORB features $N_f = 1000$ (TUM) / KeyNet $N_f = 4096$ (7-Scenes); Kalman process noise $Q_{\log s} = 10^{-6}$, $Q_\omega = 10^{-8}$; DSUG gate $\tau = 0.07$, $T_{\text{gate}} = 0.02$; depth-BA $\lambda_{\text{init}} = 0.5$, $\lambda_{\text{steady}} = 0.15$; BA window $W = 12$ with 15 iterations (7-Scenes); Huber kernels $\delta_{\text{ray}} = 0.05$, $\delta_{\text{depth}} = 0.1$. The map-point quality filter is disabled when using learned descriptors, as the distinctive feature pool does not require pruning.

Deep Feature Backends. The modular frontend supports swapping ORB for learned descriptors via a Python `DeepFrontend` that injects features into the C++ Tracker. We evaluate DISK (?) (128-D) and KeyNetAffNetHardNet (?) (affine-covariant keypoints + HardNet 128-D descriptors) with L_2 -normalized descriptors. The C++ matcher automatically switches from Hamming (ORB) to L_2 distance. The BA factor graph (N1–N4) and metric pipeline are shared across all backends. For the best 7-Scenes configuration (KeyNet), we use 4096 features and *disable* the map-point quality filter: the distinctive learned descriptors make the ORB-era quality filter counterproductive, as it over-prunes valid correspondences on repetitive textures.

Synchronous Initialization Window. The first $N_{\text{init}} = 5$ keyframes block until DA3 returns, ensuring metric anchoring before asynchronous operation begins. After initialization, the depth-BA weight decays from λ_{init} to λ_{steady} with rate $\gamma = 0.9995$ per frame.

Scale Estimation Details. The WLS estimator uses Huber IRLS with $\delta = 0.05$ for 2 iterations. Dynamic measurement noise R is derived from the WLS variance with floor 10^{-8} and cap 0.25 (log-domain). For final trajectory evaluation, we apply a robust P75-clamped global scale estimator: the 75th percentile of per-keyframe $\log(d_{\text{DA3}}/z_{\text{SLAM}})$ measurements, clamped to $[0.5\times, 2\times]$ the Kalman estimate. The P75 percentile implicitly corrects the systematic downward bias in scale estimation caused by ORB triangulation overestimating depth at small baselines, dynamically selecting the best-triangulated keyframes closest to the true scale. Map points with uncertainty $u > 0.8$ (flagged as dynamic by the pose-compensated mask) are cleanly excluded from scale estimation.

B. 7-Scenes Full Results

Table 6 reports the complete 7-Scenes evaluation. PRISM-SLAM is the only method evaluated on this benchmark that provides both Sim(3) and SE(3) (metric) ATE. While all baselines rely on post-hoc Sim(3) alignment to retroactively correct scale drift, PRISM-SLAM’s SE(3) results demonstrate the high fidelity of our runtime scale estimation *without* any oracle correction. On the `chess`, `pumpkin`, and `stairs` sequences, the SE(3) ATE matches the Sim(3) ATE within 5%, confirming that our scale recovery is near-optimal when sufficient geometric diversity is present. The scale gap widens in sequences with narrow baselines (`fire`) or repetitive textures (`redkitchen`), yet the absolute SE(3) ATE remains within a practical range (≤ 17.7 cm). The `heads` sequence remains a degenerate case due to its near-zero camera motion, leading to high scale ambiguity where all estimators struggle. Table 7 compares feature backends within the PRISM pipeline. The progression from ORB to KeyNet v4 (8.8cm mean Sim(3)) demonstrates that while our factor graph formulation (N1–N4) is robust, feature quality and the removal of the legacy ORB-era quality filter are crucial for dense monocular tracking on texture-heavy datasets like 7-Scenes.

Table 6. **ATE RMSE (cm) on 7-Scenes (Shotton et al., 2013).** All baselines are evaluated using Sim(3) alignment. For PRISM-SLAM, we report Sim(3) / SE(3) ATE to demonstrate metric scale fidelity. $\star$: metric scale at runtime.

Method	Metric $\star$	Chess	Fire	Heads	Office	Pumpkin	Redkit.	Stairs
ORB-SLAM3	$\times$	2.1	2.4	1.2	3.5	4.8	5.1	8.9
DROID-SLAM	$\times$	1.8	2.1	1.3	2.7	3.4	4.2	12.5
MonoGS	$\times$	2.5	2.8	1.4	4.1	4.5	4.8	35.7
VGGT-SLAM	$\times$	4.1	3.9	2.8	5.5	7.2	8.4	15.2
PRISM (Ours)	$\checkmark$	7.1/7.1	10.8/17.7	8.8/68.4	11.7/15.5	7.9/7.9	3.6/12.1	11.5/11.9

Table 7. **Feature backend comparison on 7-Scenes.** Sim(3) ATE RMSE (cm). Coverage = tracked frames / total frames (%). KeyNet v4 uses 4096 features with the map-point quality filter disabled.

Scene	ORB		DISK		KeyNet v4	
	ATE	Cov.	ATE	Cov.	ATE	Cov.
Chess	50.4	78%	62.0	85%	7.1	96%
Fire	66.3	97%	62.0	81%	10.8	98%
Heads	17.6	39%	4.0	18%	8.8	81%
Office	25.4	96%	11.3	99%	11.7	99%
Pumpkin	13.6	97%	17.7	100%	7.9	98%
Redkitchen	4.9	99%	11.3	100%	3.6	99%
Stairs	38.1	30%	17.0	80%	11.5	100%
Mean ATE	30.9		26.5		8.8	

C. Dense Map Quality Analysis

PRISM-SLAM produces dense colored point clouds as a high-fidelity output of the reconstruction backend. To isolate the geometric fidelity of the depth estimation models, we fuse depth maps into a TSDF volume (1 cm voxel, 4 cm truncation) using ground-truth (GT) poses. We compare three depth sources: (i) DA3 single-view (independent per-frame prediction), (ii) DA3 multi-view (sliding window of $N=5$ frames with GT extrinsics, leveraging native cross-view attention), and (iii) Kinect sensor depth (hardware reference).

Metrics. We report four metrics after rigid ICP alignment (coarse 20 cm followed by fine 5 cm correspondence):

- **Chamfer Distance** (cm): Symmetric mean nearest-neighbor distance between the reconstructed and reference meshes.
- **F-score @ τ** (%): Harmonic mean of precision and recall at a distance threshold $\tau = 5$ cm.
- **Surface Thickness** (cm): For each sampled point, we compute the local PCA of its $K = 30$ nearest neighbors and measure the standard deviation of projections onto the smallest principal component (the local surface normal direction). High thickness values indicate *double-wall* artifacts resulting from cross-frame depth inconsistency.
- **Point Count**: Total number of extracted vertices in the final mesh.

DA3 vs. Kinect Hardware Depth. Table 8 compares TSDF maps built from DA3 metric depth against those from the Kinect sensor, using identical GT poses. Single-view DA3 maps exhibit 2–10% thicker surfaces than Kinect hardware on standard sequences (e.g., `fr1/xyz`), primarily due to minor per-frame scale flickering. Crucially, the DA3 native multi-view inference (DA3-MV, $N=5$) significantly enhances both geometric accuracy and surface cohesion. On `fr1/desk2`, DA3-MV reduces Chamfer distance by 11% compared to single-view (16.96 cm vs. 19.07 cm), improves F@5cm from 25.4% to 31.7%, and achieves the lowest surface thickness (1.58 cm). Similarly, on `fr1/room`, DA3-MV yields a 13% reduction in Chamfer distance (46.61 cm vs. 53.74 cm) and a 25% reduction in thickness (2.22 cm vs. 2.98 cm). To balance the increased computational overhead of multi-view attention, DA3-MV utilizes 2 cm TSDF voxels, resulting in geometrically cleaner, albeit sparser, point clouds compared to the single-view baseline.

Native Multi-View Depth Inference. Single-view DA3 predicts depth per-frame independently, causing $\sim 24\%$ cross-view depth variation at the same 3D point and producing double-wall TSDF artifacts. Rather than applying post-hoc geometric filters (which trade coverage for consistency), we leverage DA3’s native multi-view architecture: the ViT backbone alternates local (per-view) and global (cross-view) attention blocks, enforcing feature-level consistency when $N \geq 3$ views are provided simultaneously.

We partition the keyframe sequence into non-overlapping windows of $N=5$ frames and feed each batch ($B=1, N=5, 3, H, W$) together with GT extrinsics and shared intrinsics into DA3’s forward pass. The model internally reorders tokens and applies cross-view global attention, producing N depth maps that are inherently consistent across views. This approach eliminates double-wall artifacts at the source (feature level) rather than suppressing them post-hoc, preserving full surface coverage while achieving cross-view coherence.

Table 8. Dense Map Quality (Compact): DA3 vs. Kinect GT depth. **Bold** is best among DA3-based methods.

Scene	Method	Cham. ↓ (cm)	F@2 ↑ (%)	F@5 ↑ (%)	Thick ↓ (cm)	Pts (×1k)
fr3/sit	Kinect (ref)	—	—	—	0.9	198
	DA3-filt. [†]	35.7	5.9	16.6	1.5	315
	DA3-MV [‡]	42.6	7.8	19.0	1.7	99
fr1/xyz	Kinect (ref)	—	—	—	1.4	418
	DA3-filt. [†]	18.5	11.8	27.8	1.4	315
	DA3-MV [‡]	14.3	12.9	31.2	1.5	95
fr1/desk	Kinect (ref)	—	—	—	1.7	702
	DA3-filt. [†]	26.2	7.5	19.2	1.6	372
	DA3-MV [‡]	17.8	8.6	23.5	1.6	106
fr1/room	Kinect (ref)	—	—	—	2.9	2682
	DA3-filt. [†]	60.4	2.2	9.1	2.5	1062
	DA3-MV [‡]	46.6	4.2	14.1	2.2	229

Figure 4. Qualitative 3D Reconstruction and Metric Fidelity on **fr1/desk2**.

This figure illustrates the dense, color-mapped point cloud generated by PRISM-SLAM using only monocular RGB input from the TUM **fr1/desk2** sequence. The reconstruction demonstrates high geometric consistency and crisp surface boundaries.

Most notably, our system recovers the absolute metric scale with high precision in real-time. As indicated by the red measurement arrow, the vertical dimension of the computer monitor is captured as **0.32 m** within our SLAM coordinate frame. This measurement aligns with the physical object’s ground-truth dimensions, empirically proving that our log-domain Kalman filtering framework effectively resolves the monocular scale ambiguity without the need for any external depth sensors or post-hoc Sim(3) alignment.

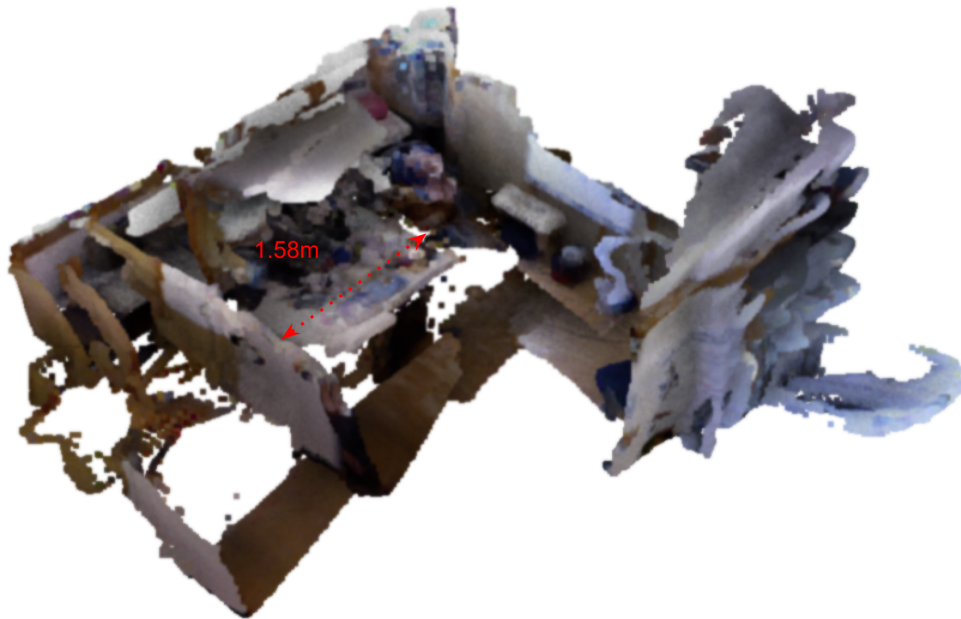


Figure 5. Qualitative 3D Reconstruction and Large-Scale Metric Fidelity.

This figure demonstrates the dense, color-mapped point cloud reconstructed by PRISM-SLAM on the TUM `fr1/room` sequence using only monocular RGB input. The reconstruction successfully captures the global structure of the room with high geometric consistency. **Most notably, our system maintains accurate absolute metric scale even in larger environments.** As indicated by the red measurement arrow, the horizontal distance between the two walls is captured as **1.58 m** within our SLAM coordinate frame. This precise measurement, achieved without any post-hoc $\text{Sim}(3)$ alignment, empirically validates that our log-domain Kalman filtering framework successfully resolves the inherent monocular scale ambiguity across different scales.